

Quality Inspection Tracker

Project Synopsis

Repository: <https://github.com/niravparmar197/quality-inspection-tracker>

Objective

Quality Inspection Tracker is a mobile-first web application that lets shop-floor supervisors at textile manufacturing plants log, track, and resolve quality defects digitally. It replaces paper registers and delayed spreadsheet entry with a tool usable directly on a phone at the point of inspection.

Features

- Log a new inspection: date, machine/line ID, defect type, severity, and optional remarks
- View all inspections in a filterable, sortable list (by severity, status, and date range)
- Mark an inspection as Resolved with a mandatory resolution note
- Dashboard summary of Open and Resolved inspection counts by severity
- User registration and JWT-based login; all inspection data is protected
- Offline support: inspections logged without connectivity are queued locally and synced automatically once back online
- Mock SAP integration: an inbound webhook lets an external system create inspections, and the app notifies an outbound endpoint whenever inspections are created or resolved
- Responsive, mobile-first UI usable on phone screens as narrow as 390px

Technology Used

- **Backend:** NestJS 11, Prisma ORM 7, SQLite, JWT authentication (@nestjs/jwt, passport-jwt, bcrypt), Swagger / OpenAPI docs
- **Frontend:** React 19, TypeScript, Vite, Material UI, Tailwind CSS, React Hook Form, Axios, localforage
- **Deployment:** Docker and Docker Compose, API and frontend as separate containers

Architecture / Workflow

The React frontend (a single-page app) calls a NestJS REST API over HTTP, authenticated with a JWT bearer token issued at login. The API validates and persists data through Prisma into a local SQLite database. If the device is offline when an inspection is logged, the entry is queued in the browser and automatically submitted once the connection returns. An inbound webhook endpoint lets an external system (e.g. SAP) create inspections directly, and the backend separately notifies a mock outbound endpoint whenever an inspection is created or resolved, demonstrating both directions of a plant-system integration.

How to Set Up

Option A — Docker (recommended):

```
docker compose up --build
```

Frontend: <http://localhost:5173> Backend: <http://localhost:3000> Swagger:

<http://localhost:3000/api>

Option B — Local development:

```
cd server
npm install
npx prisma generate
npx prisma migrate dev
npm run start:dev
```

```
cd client
npm install
npm run dev
```

No seeded account exists — register a new user via the app's Register page or POST /auth/register.

Conclusion

The Quality Inspection Tracker digitizes a manual, paper-based defect-logging process into a fast, structured, and searchable system. The expected outcome is fewer reporting delays, more accurate and timely defect records, and a foundation that can be extended toward closer integration with plant systems such as SAP as the tool matures.